STEP 0

Prerequisites & Setup

Install

- **Flutter 3.24+** — flutter.dev
- Run `flutter doctor` — all green
- IDE: VS Code or Android Studio
- Android / iOS device or emulator

Clone & Run

```
# Clone the workshop repo
git clone https://github.com/DenisovAV/\
  workshop-flutter-gemma-hybrid-ai.git

cd workshop-flutter-gemma-hybrid-ai
git checkout step-00-starter

# Run the starter
flutter pub get
flutter run
```

You should see an echo chat — messages come back with "AI service not connected yet"

STEP 1

Firebase Setup

Firebase Console

- Go to **console.firebase.google.com**
- Create project: `workshop-hybrid-ai`
- Navigate to **AI Logic** in sidebar
- Enable **Gemini API**

Configure

```
# Install FlutterFire CLI
dart pub global activate flutterfire_cli

# Configure your app
flutterfire configure \
  --project=YOUR_PROJECT_ID
```

Uncomment Dependencies

```
# pubspec.yaml
firebase_core: ^4.4.0
firebase_ai: ^3.8.0
```

STEP 2

Cloud Chat — FirebaseAIService

```dart
import 'package:firebase_ai/firebase_ai.dart';

class FirebaseAIService implements AIService {
  GenerativeModel? _model;

  Future<void> initialize() async {
    final ai = FirebaseAI.googleAI();
    _model = ai.generativeModel(model: 'gemini-2.5-flash');
  }

  Stream<String> generateResponseStream(String prompt) async* {
    final stream = _model!.generateContentStream([Content.text(prompt)]);
    await for (final chunk in stream) {
      if (chunk.text != null) yield chunk.text!;
    }
  }
}
```

⚡ main.dart

Add `Firebase.initializeApp()` before `runApp()`

☰ Streaming

`generateContentStream` returns tokens one by one for real-time UI

STEP 3

Platform Setup

iOS

```
# Podfile
platform :ios, '16.0'
use_frameworks! :linkage => :static

# Runner.entitlements
extended-virtual-addressing
increased-memory-limit
```

Android

```
<!— AndroidManifest.xml —>
<uses-native-library
  android:name=
    "libOpenCL.so"
  android:required=
    "false" />
<!— +libOpenCL-car.so —>
<!— +libOpenCL-pixel.so —>
```

Web

```
<!— index.html —>
<script type="module">
import {
  FilesetResolver,
  LlmInference
} from
  '@mediapipe/tasks-genai
   @latest';
</script>
```

HuggingFace Token

huggingface.co/settings/tokens → New token → Read access

Dependency

Uncomment `flutter_gemma: ^0.12.4` in pubspec.yaml

STEP 4

Local Inference — LocalAIService

```
FlutterGemma.initialize();

// Download model (~500 MB)
await FlutterGemma.installModel(modelType: ModelType.gemmaIt)
    .fromNetwork(modelUrl, token: hfToken)
    .withProgress((p) => onProgress(p / 100)).install();

// Create model + chat session
final model = await FlutterGemma.getActiveModel(
  maxTokens: 1024, preferredBackend: PreferredBackend.gpu);
_chat = await model.createChat();

// Generate response
await _chat.addQuery(Message.text(text: prompt, isUser: true));
await for (final chunk in _chat.generateChatResponseAsync()) {
  if (chunk is TextResponse) yield chunk.token;
}
```

⚠️ **isUser: true** is critical! Default is `false` → empty responses

UI: SegmentedButton

Cloud / Local mode picker

STEP 5

Hybrid Strategy — HybridAIService

4 Strategies

localFirst
Local → Cloud

cloudFirst
Cloud → Local

localOnly
Offline mode

cloudOnly
Cloud only

Fallback Pattern

```
case AIStrategy.localFirst:
  try {
    yield* local.generateResponseStream(prompt);
  } catch (e) {
    yield* cloud.generateResponseStream(prompt);
  }
```

UI: 4-option SegmentedButton

Cloud | Local | Local+Cloud | Cloud+Local — `yield*` forwards the entire stream, `try/catch` switches to fallback

STEP 6

Embeddings + VectorStore

Install & Load

```
// Install embedding model
await FlutterGemma.installEmbedder()
  .modelFromNetwork(modelUrl, token: hfToken)
  .tokenizerFromNetwork(tokenizerUrl, token: hfToken)
  .install();

// Initialize VectorStore (SQLite)
await FlutterGemmaPlugin.instance
  .initializeVectorStore('rag.db');

// Load docs (auto-embeds!)
await FlutterGemmaPlugin.instance.addDocument(
  id: 'paris',
  content: description,
  metadata: jsonEncode({'city': 'Paris'}),
);
```

How Embeddings Work

```
"cat"    →  [0.2, 0.8, 0.1, ...]
"kitten" →  [0.3, 0.7, 0.2, ...] ~ similar!
"car"    →  [0.9, 0.1, 0.7, ...] ~ different
```

10 Cities Loaded

Paris, Tokyo, New York, Barcelona, Istanbul, Sydney, Rio, Marrakech, Prague, Singapore

STEP 7

RAG: Query + Generation

RAG Pipeline

Query → Embed → Search → Context → Generate

```
// Search (auto-embeds query)
final results = await plugin.searchSimilar(
  query: prompt, topK: 3, threshold: 0.5,
);

// Build augmented prompt
final context = results
    .map((r) => r.content).join('\n\n');
final augmented =
  'Based on: $context\n\nAnswer: $query';

// Generate with context
await _chat.addQuery(
  Message.text(text: augmented, isUser: true));
```

Demo Queries

Query	Source
Cherry blossoms?	Tokyo
Mediterranean food?	Barcelona
Ancient architecture?	Prague, Istanbul
Beach vacation?	Rio, Sydney

UI

RAG toggle (Switch) + Sources banner showing matched cities